7조 - 팀 졸지마라

딥러닝 기반 졸음 감지
도로 환경 통합 안전 제어 시스템

팀장 : 장태희
팀원 : 한소영, 김형진

목차

01 프로젝트 개요

02 프로젝트 구성 및 결과

03 AI 모델 학습 구조

04 하드웨어 설계 및 구현

05 트러블 슈팅

06 부록

팀원 역할 분담

팀장 장태희



차량 인식 및 AI 학습
프로젝트 일정 조율

한소영



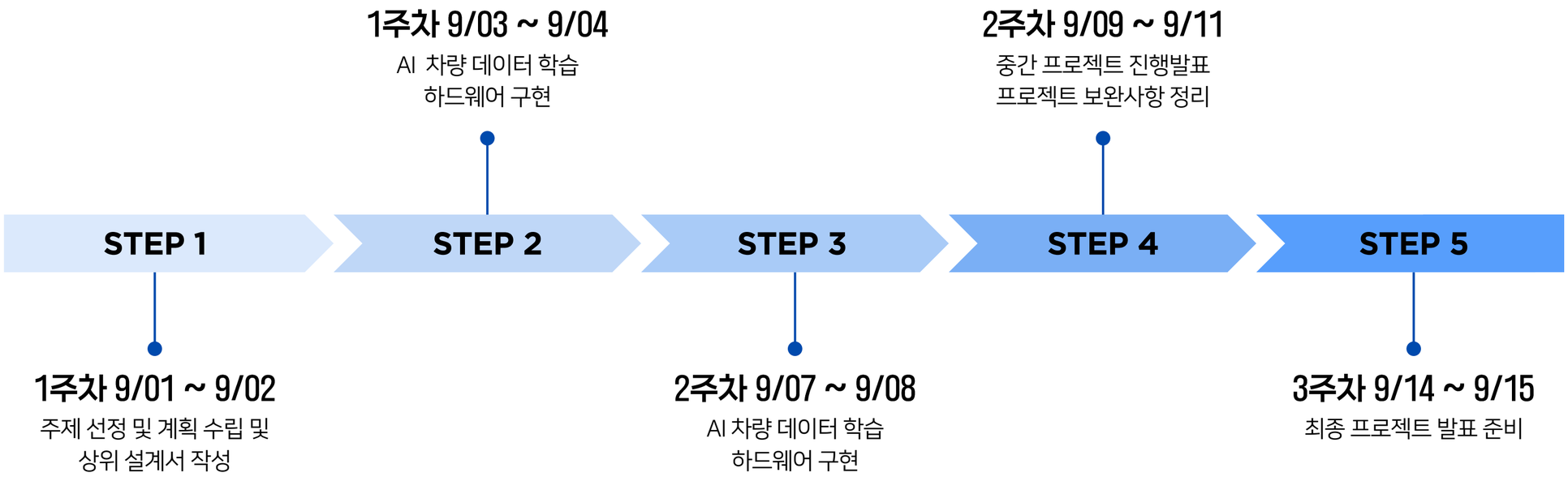
졸음(눈 감음), 하품
경고 발생 및 GUI 구현

김형진



STM32 하드웨어 구현 및
통신 프로토콜 작성

프로젝트 진행 일정


1주차 9/03 ~ 9/04
AI 차량 데이터 학습
하드웨어 구현
2주차 9/09 ~ 9/11
중간 프로젝트 진행발표
프로젝트 보완사항 정리
STEP 1
STEP 2
STEP 3
STEP 4
STEP 5
1주차 9/01 ~ 9/02
주제 선정 및 계획 수립 및
상위 설계서 작성
2주차 9/07 ~ 9/08
AI 차량 데이터 학습
하드웨어 구현
3주차 9/14 ~ 9/15
최종 프로젝트 발표 준비

주제 선정 배경 및 목적

기존 문제 상황

고속도로 교통사고 치사율 1위인 졸음운전은
단순 시각/청각 경고만으로는 운전자의 각성을 유도하기 어려움.

밀폐된 차량 내부의 이산화탄소 농도 증가 및
시야 확보가 어려움

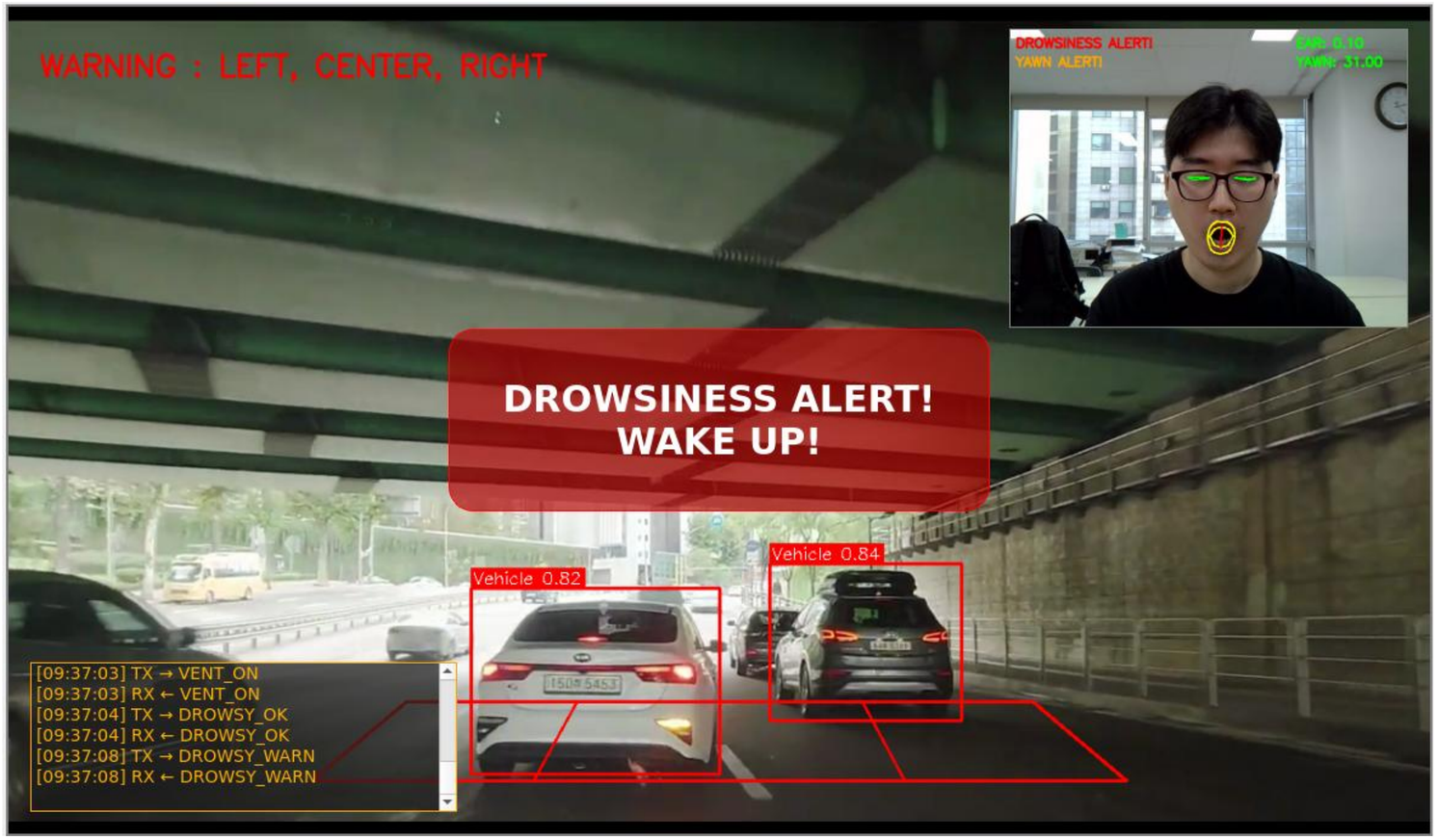
터널 진·출입 구간, 전방 급제동 상황이 결합될 때
대형 연쇄 추돌 사고로 직결됨.

제시되는 해결방안

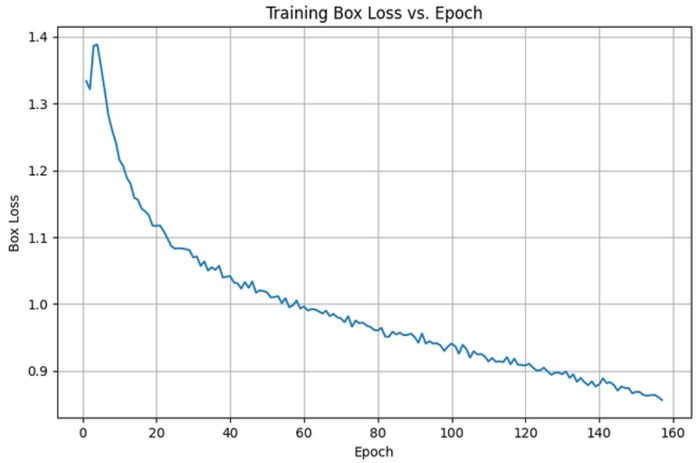
- 운전자 상태 실시간 모니터링
- 전방 도로 및 환경 실시간 인지
- 하드웨어 제어(창문, 에어컨 등) 통합 제어
- 분산 모니터링 구축 (GUI로 모니터링)

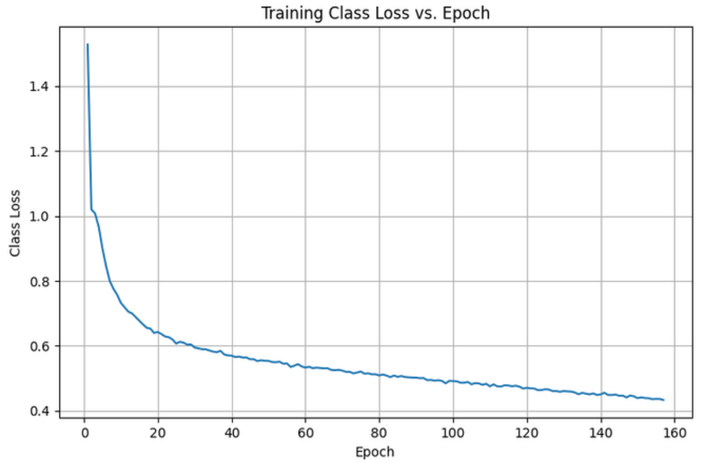
**본 시스템은 Jetson Orin Nano 기반의 AI 관제 시스템과 STM 32 기반의 임베디드 제어기,
그리고 PC 렌더링 관제 GUI가 UART 통신으로 결합된
운전자 졸음 방지 및 도로 환경(터널/차간거리) 통합 안전 제어 시스템**

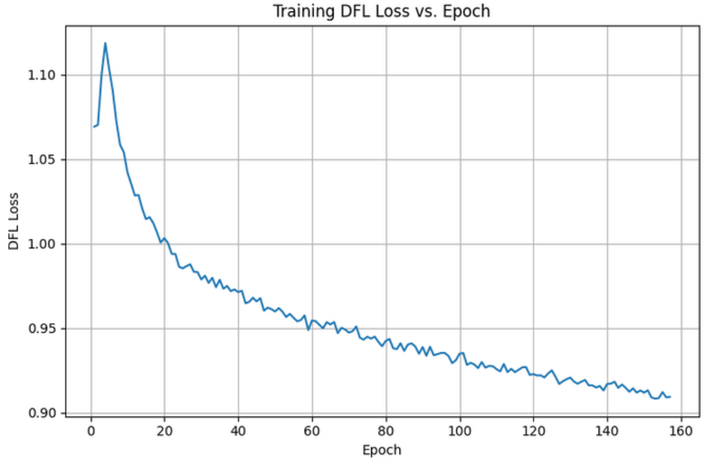
제작 결과

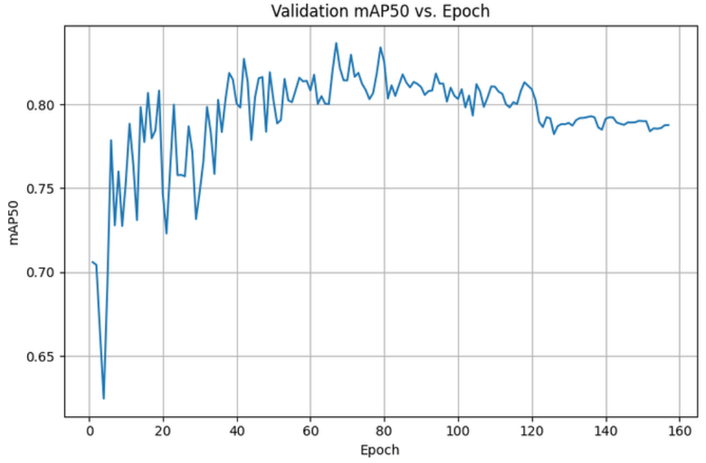

WARNING : LEFT, CENTER, RIGHT
DROWSINESS ALERT!
YAWN ALERT!
EAR: 0.10
YAWN: 31.00
DROWSINESS ALERT!
WAKE UP!
Vehicle 0.82
Vehicle 0.84
[09:37:03] TX → VENT_ON
[09:37:03] RX ← VENT_ON
[09:37:04] TX → DROWSY_OK
[09:37:04] RX ← DROWSY_OK
[09:37:08] TX → DROWSY_WARN
[09:37:08] RX ← DROWSY_WARN

AI 모델 학습 결과









지표별 세부 분석

- Training Box / DFL Loss: 초반 급격한 하락 후 160 에폭까지 꾸준히 우하향(Box: 1.39 → 0.86, DFL: 1.12 → 0.91)하며 경계 상자 위치 추정 성능이 지속적으로 향상되고 있음을 확인할 수 있음.
- Training Class Loss: 초기 1.53에서 10 에폭 이내로 급격히 감소한 뒤 0.43 수준까지 안정적으로 수렴하여 클래스 분류 학습이 원활하게 진행됨을 확인
- Validation mAP50: 에폭 60~80 구간에서 최고 성능(약 0.83~0.84)을 달성했으나, 에폭 100 이후 0.78~0.79 수준으로 소폭 하락하며 정체 됨. 원래 200Epoch 동작이나, 과적합이 나오기 전에 학습이 멈춤.
- Ultralytics YOLO의 patience 기본값(Default)은 100으로, 진행 중인 학습에서 최고 성능(mAP50)이 약 60 에폭 전후에서 관찰된 뒤, 성능 개선이 이루어지지 않자 100 에폭이 경과한 160 에폭 부근에서 Early Stopping이 정상적으로 발동된 상황.

대표 시나리오

운전자 졸음 발생 시

눈을 2초 이상 감으면 알람이 작동하고,
눈을 5초 이상 뜨고 있으면 해제된다.

UART 통신



STM32

Jetson에서 수신한 명령에 따라
하드웨어를 제어

UART 통신

터널 진입 및 탈출

터널 진입 시 창문을 닫고 에어컨을 가동하며,
탈출 시 창문을 열고 에어컨을 정지시킨다.

하품 발생시

하품을 감지하면 환기 시스템을 5초간
작동시킨다.

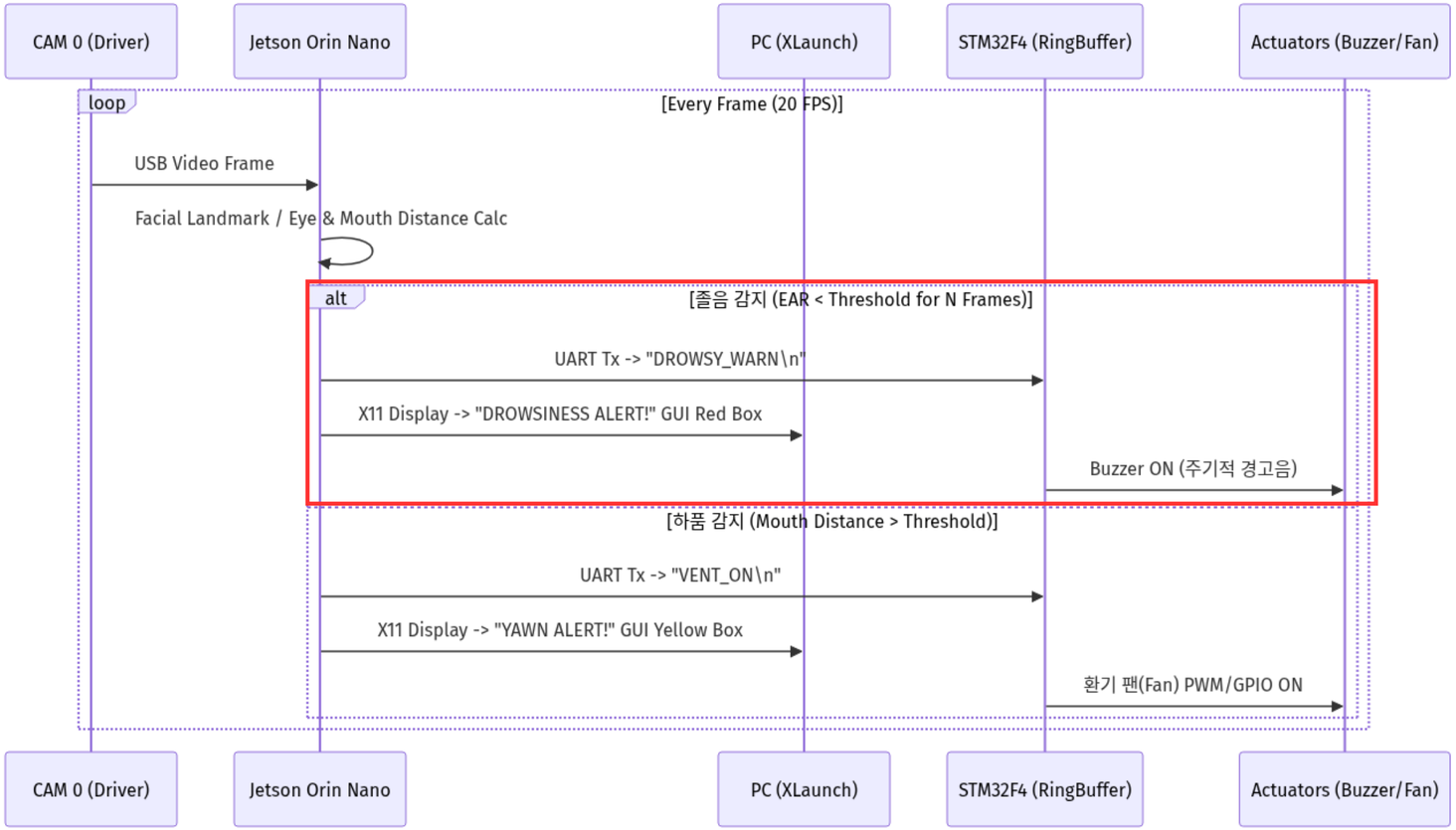
UART 통신

UART 통신

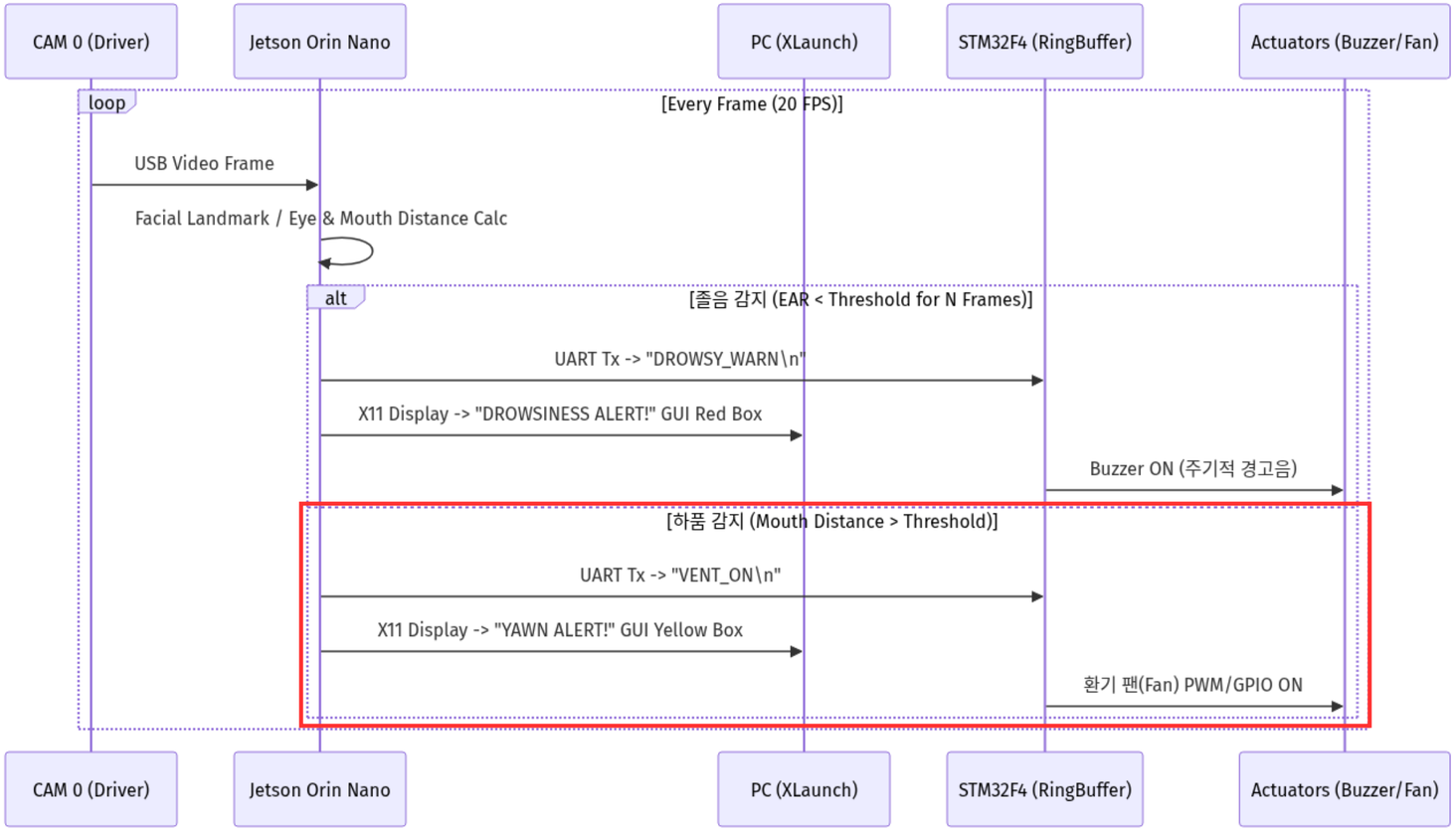
차량 근접 시

접근 차량의 방향에 따라 LED를 동작
시키고, 차량이 멀어지면 LED를 끈다.

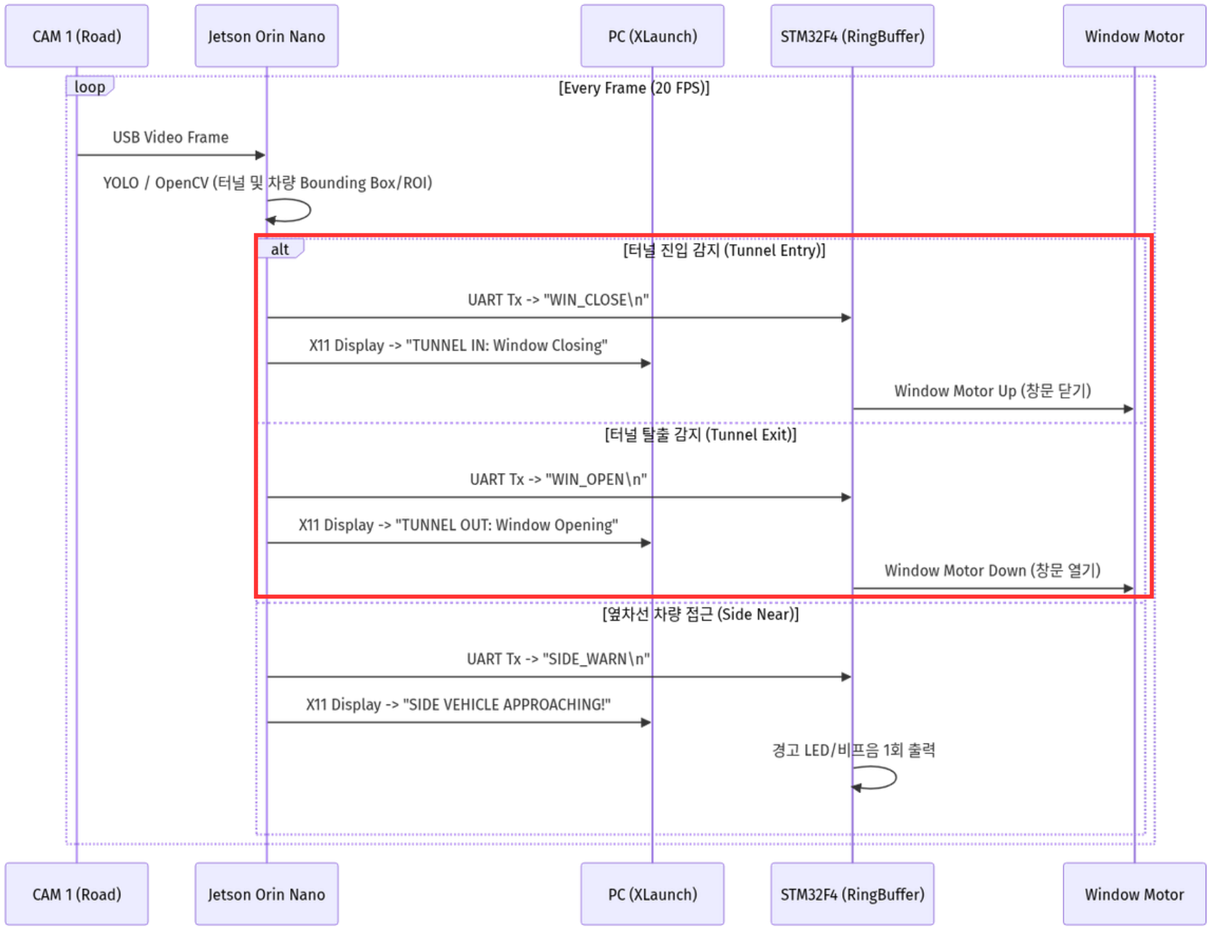
전체 동작 흐름 및 시스템 구성도(운전자 화면)



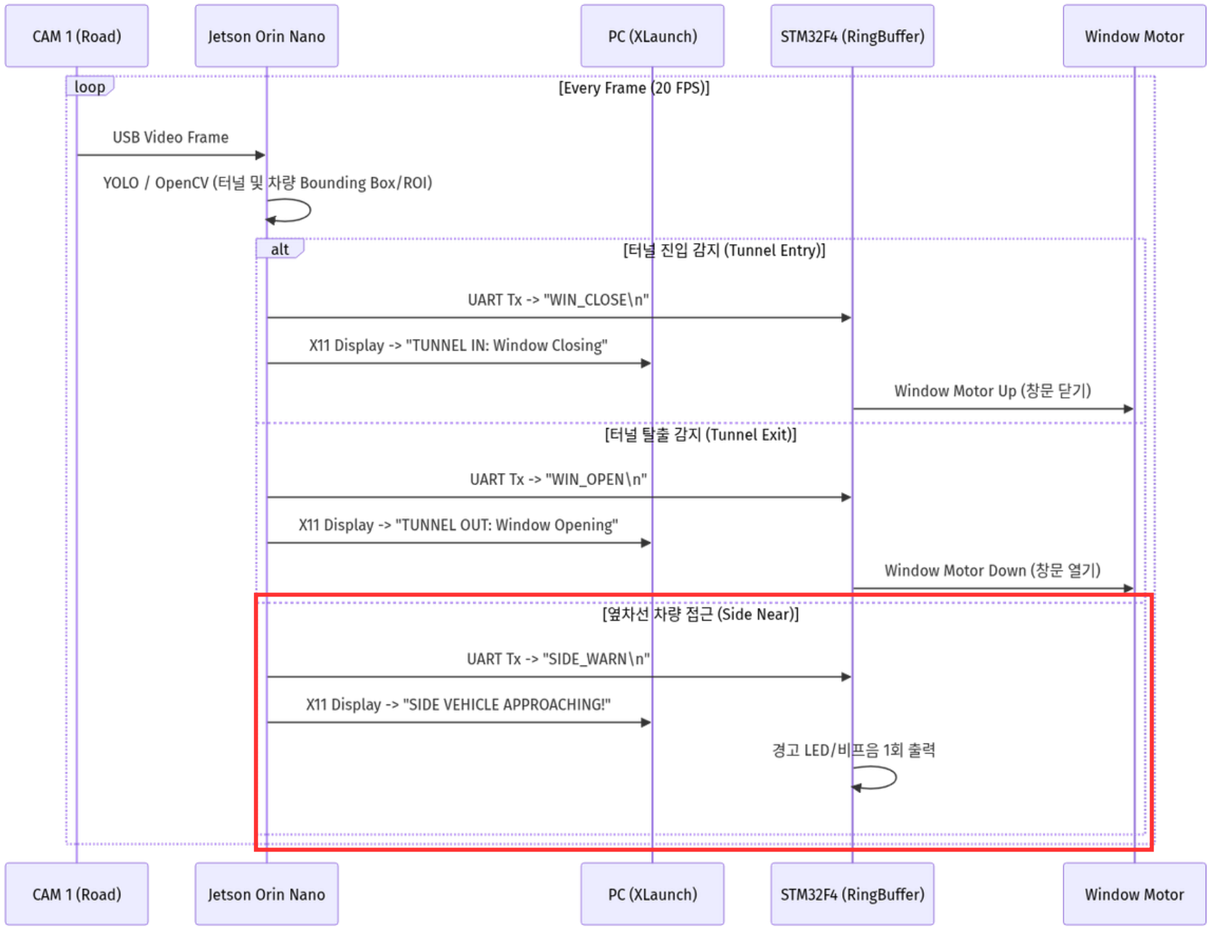
전체 동작 흐름 및 시스템 구성도(운전자 화면)



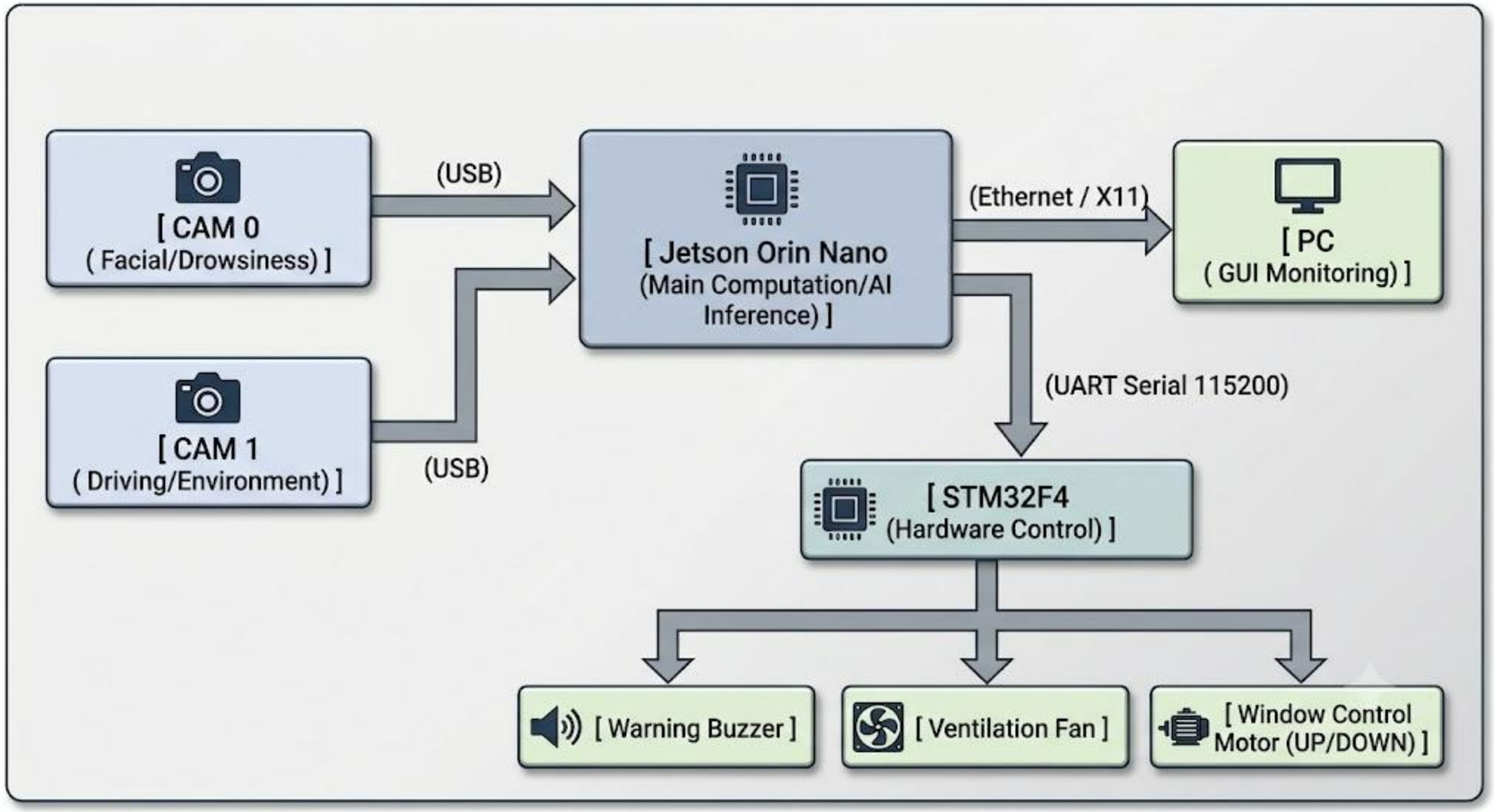
전체 동작 흐름 및 시스템 구성도(전방화면)



전체 동작 흐름 및 시스템 구성도(전방화면)



시스템 아키텍처


[CAM 0
(Facial/Drowsiness)]
(USB)
[Jetson Orin Nano
(Main Computation/AI
Inference)]
(Ethernet / X11)
[PC
(GUI Monitoring)]
[CAM 1
(Driving/Environment)]
(USB)
(UART Serial 115200)
[STM32F4
(Hardware Control)]
[Warning Buzzer]
[Ventilation Fan]
[Window Control
Motor (UP/DOWN)]

MediaPipe 기반 운전자 상태 감지



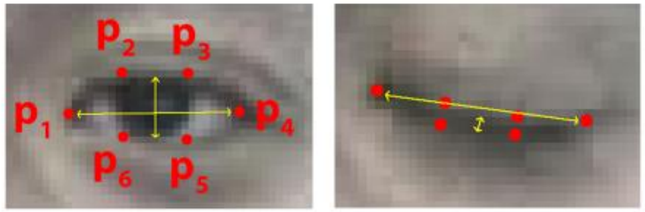
MediaPipe Face Mesh를 활용,
1) 눈 감김(졸음), 2) 입 벌림(하품), 3) 얼굴 미감지 상황추적

MediaPipe Face Mesh는 Google에서 개발한
실시간 3D 얼굴 랜드마크 검출 프레임워크

단일 RGB 카메라 입력(2D 이미지)만으로 얼굴 영역에서
총 468개(개선 버전은 눈동자 10개 포함 478개)의 3D
좌표(x, y, z)를 추정.

MediaPipe 기반 운전자 상태 감지

$$EAR = \frac{||p_2 - p_6|| + ||p_3 - p_5||}{2 \cdot ||p_1 - p_4||}$$



눈 종횡비(EAR, Eye Aspect Ratio) 계산

눈의 수직 거리 2개(위아래 눈꺼풀)와 수평 거리 1개(눈꼬리 간격)의 비율을 계산.
눈을 뜨면 값이 높아지고, 눈을 감을수록 0.00에 가까워지는 특성을 이용해 눈 뜸/감김을 수치화.

EYE_AR_THRESH = 0.25
: 눈 감김을 판단하는 임계값(EAR), EAR 값이 이보다 작아지면 눈을 감은 것으로 간주

YAWN_THRESH = 25
: 하품을 판단하는 입 수직 거리 임계값(픽셀 단위)

MediaPipe 기반 운전자 상태 감지

졸음 감지

- 눈 좌표를 이용한 EAR 계산

```python
def cam0_calculate_ear(eye_points):
    """6개의 눈 좌표로 EAR(눈 종횡비)을 계산한다."""
    #눈의 세로 거리 2개
    vertical_1 = dist.euclidean(eye_points[1], eye_points[5])
    vertical_2 = dist.euclidean(eye_points[2], eye_points[4])

    #눈의 가로 거리
    horizontal = dist.euclidean(eye_points[0], eye_points[3])

    if horizontal == 0:
        return 0.0

    return (vertical_1 + vertical_2) / (2.0 * horizontal)
```

- EAR이 임계값보다 작으면 눈 감김으로 판단

```python
eyes_closed = ear < EYE_AR_THRESH
```

```python
if not is_warning_active:
    # 경고 발생 전에는 눈을 연속으로 감은 시간을 측정한다.
    _eyes_open_since = None
    _recovery_blink_since = None

    if eyes_closed:
        if _eyes_closed_since is None:
            _eyes_closed_since = current_time

        if (
            current_time -  _eyes_closed_since
            >= EYES_CLOSED_WARNING_SECONDS #2.0
        ):
            trigger_warning = True
    else:
        _eyes_closed_since = None
```

- 눈 감김이 2초 이상 지속되면 졸음 경고 발생
- 경고 후 눈을 뜬 상태가 5초 유지 되면 정상 상태로 복귀
- 회복 중 0.5초 이내의 짧은 깜빡임은 허용

하품 감지

- 입술 사이의 수직 거리 계산

```python
top_lip = landmarks[MOUTH_TOP_BOTTOM[0]]
bottom_lip = landmarks[MOUTH_TOP_BOTTOM[1]]
lip_distance = abs(top_lip[1] - bottom_lip[1])
```

- 입술 사이의 수직 거리가 임계값보다 크면 하품으로 판단

```python
YAWN_THRESH = 25
```

```python
if lip_distance > YAWN_THRESH:
    status_info["yawn_detected"] = True
```

MediaPipe 기반 운전자 상태 감지



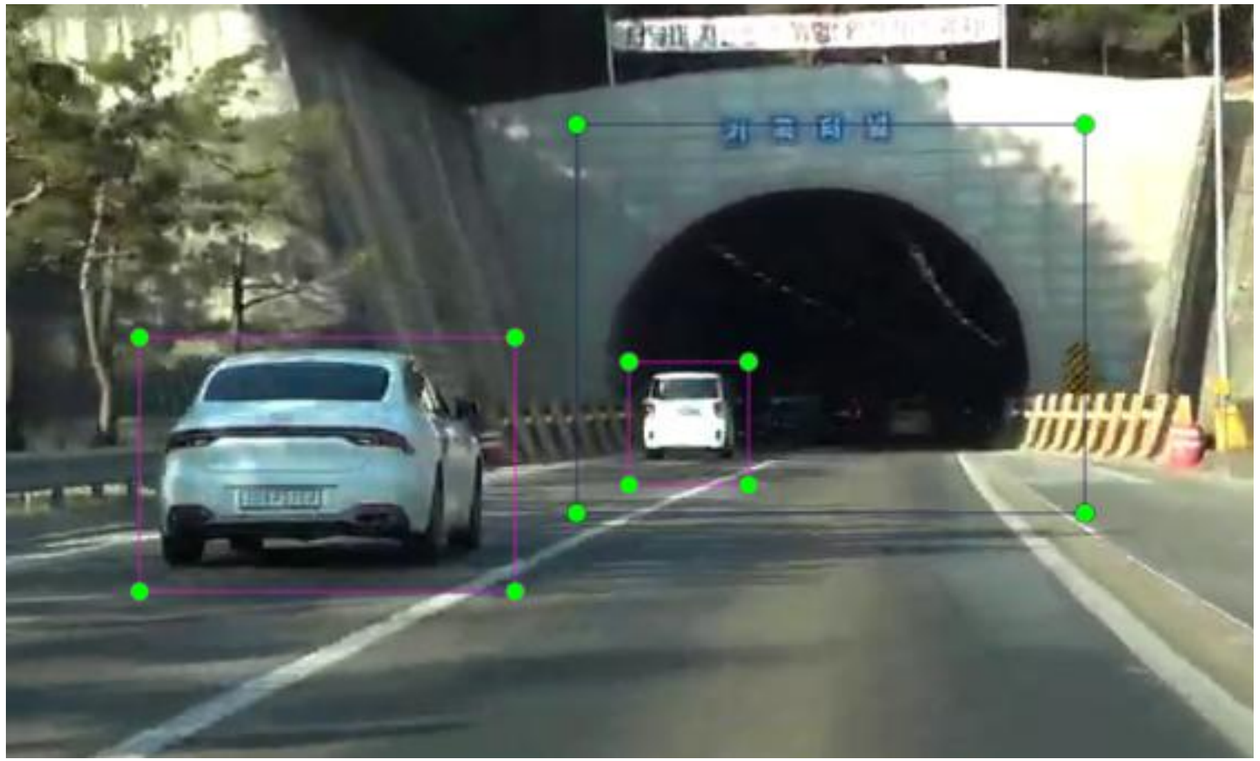


MediaPipe 기반 운전자 상태 감지





AI 모델 학습 구조



YOLO11n (You Only Look Once) 모델
- 지도 학습(Supervised Learning) 방식

학습데이터
- train 7204장 (라벨링이미지 6000 배경이미지 1204)
- validation 892장 (라벨링이미지 820 배경이미지 72)

AI 모델 학습 구조

```
yolo.train(data=DATA, epochs=200, batch=16,
           imgsz=320,
           device=0,
           workers=4,
           project=project_root,
           degrees=5.0,
           flipud=0.0,
           fliplr=0.5,
           mosaic=1.0,
           name='rps_yolo11n')
```

파라미터	설정값	주요 역할 및 의미
data	DATA	**데이터셋 설정 파일 경로**: 학습에 사용할 이미지 경로, 클래스 이름, 개수 등이 정의된 .yaml 파일
epochs	200	**전체 데이터 학습 횟수**: 전체 데이터셋을 총 200번 반복하여 학습함.
batch	16	**배치 크기 (Batch Size)**: 1회 가중치 업데이트 시 모델에 동시에 입력 미지 개수 (GPU 메모리에 영향)
imgsz	320	**입력 이미지 해상도**: 모델 입력 시 이미지를 320x320 픽셀로 리사이즈 습 (속도 향상, 미세 객체 감지율 감소)
device	0	**할당할 GPU 번호**: 연산에 사용할 CUDA 디바이스 번호 (0번 첫 번째 용)
workers	4	**데이터 로딩 멀티프로세스 개수**: CPU에서 이미지를 불러오고 전처리 드/프로세스 수
project	project_root	**결과 저장 루트 디렉토리**: 학습 가중치(best.pt), 그래프, 로그 등이 저 위 폴더 경로
degrees	5	**이미지 회전 증강 (Data Augmentation)**: 학습 시 이미지를 랜덤하 도 ~ +5.0도 범위 내에서 회전함 설정
flipud	0	**상하 반전 확률 (Vertical Flip)**: 이미지를 위아래로 뒤집을 확률 (0. 반전 안 함)
fliplr	0.5	**좌우 반전 확률 (Horizontal Flip)**: 이미지를 좌우로 거울 반전시 (50% 확률로 적용)
mosaic	1	**모자이크 데이터 증강 (Mosaic Augmentation)**: 4장의 이미지를 무작위 합성하는 기법 적용 확률 (1.0은 100% 적용)
name	'rps_yolo11n'	**실행 결과 세부 폴더명**: project 경로 내부에 생성되는 결과 저장용 서 이름 (project_root/rps_yolo11n)

AI 모델 학습 구조



TensorRT 엔진 사용:
Fast Inference를 수행함. YOLO의 PyTorch 모델(.pt)을 NVIDIA GPU 전용으로 컴파일한 최적화 바이너리.

ByteTrack 통합:
바운딩 박스 위치만 찾는 것이 아니라, 각 차량 및 터널 객체에 고유한 track_id를 부여하여 프레임 간 동일 객체를 추적함.

- 객체 수 및 프레임 간 연속성 관리 (중복 카운팅 방지)
- 깜빡임(Flickering) 및 탐지 누락 보완
- Low-Confidence(낮은 신뢰도) 객체 유실 방지

ROI(Region of Interest) 방식 탐지:
탐지된 차량 바운딩 박스의 특정 지점을 추출함.
해당 점이 좌/중앙/우 ROI 다각형 내부에 위치하는지 기하학적으로 검사.
차량이 영역 안에 들어오면 in_left, in_center, in_right 플래그를 True로 설정.

AI 모델 학습 구조

- ByteTrack 통합 및 모델 추적 수행

```
results = self.model.track(
    frame,
    persist=True,
    tracker="bytetrack.yaml",
    classes=[CLASS_VEHICLE, CLASS_TUNNEL_ENTRANCE, CLASS_TUNNEL_EXIT],
    conf=0.45,
    verbose=False,
    imgsz=320
)[0]
```

- 고유 track_id 매핑 및 프레임 간 연속성 관리 (중복 카운팅 방지)

```
if results.boxes.id is not None:
    track_ids = results.boxes.id.int().cpu().numpy()
else:
    track_ids = [-1] * len(boxes)
```

- 깜빡임(Flickering) 및 탐지 누락 보완 (MAX_MISS 카운팅)

```
for track_id, info in self.tracked_roi_vehicles.items():
    box = info["last_box"]
    conf = info["last_conf"]

    if track_id in current_frame_roi_track_ids:
        info["miss_count"] = 0
    else:
        info["miss_count"] += 1

    if info["miss_count"] <= self.MAX_MISS_VEHICLE:
        in_danger = info["in_left"] or info["in_center"] or info["in_right"]
        if info["in_left"]: flag_left = True
        if info["in_center"]: flag_center = True
        if info["in_right"]: flag_right = True

        color = (0, 0, 255) if in_danger else (255, 100, 0)
        label_text = f"Vehicle {conf:.2f}"
        self.draw_bbox(display_frame, box, label_text, color)
    else:
        expired_vehicle_ids.append(track_id)
```

AI 모델 학습 구조

- 차량 바운딩 박스 좌표 및 중앙 탐지 영역 내포 검사 (cv2.pointPolygonTest)

```
elif cls_id == CLASS_VEHICLE:
    box_coords = (x1, y1, x2, y2)

    # 1. 가운데 ROI: 하단 중앙점 OR 아래에서 1/4 지점 검사
    vehicle_bottom_center = (int((x1 + x2) / 2), y2)
    vehicle_quarter_center = (int((x1 + x2) / 2), int(y1 + (y2 - y1) * 0.75))

    in_center_bottom = cv2.pointPolygonTest(center_roi, vehicle_bottom_center, False) >= 0
    in_center_quarter = cv2.pointPolygonTest(center_roi, vehicle_quarter_center, False) >= 0

    in_center = in_center_bottom or in_center_quarter

    # 2. 양옆 ROI: 영역 일부 겹침(Intersection) 검사
    in_left = check_bbox_roi_intersection_opencv(box_coords, left_roi)
    in_right = check_bbox_roi_intersection_opencv(box_coords, right_roi)

    if in_center or in_left or in_right:
        effective_id = track_id if track_id != -1 else f"temp_{x1}_{y1}"
        current_frame_roi_track_ids.add(effective_id)

        self.tracked_roi_vehicles[effective_id] = {
            "miss_count": 0,
            "last_box": (x1, y1, x2, y2),
            "last_conf": conf,
            "in_center": in_center,
            "in_left": in_left,
            "in_right": in_right,
        }
```

탐지 구간을 총 3개 구역으로 분리 (좌, 중, 우)하여 각각의 다른 검출 조건을 부여함.

중앙의 경우 바운딩 박스의 하단 변의 중앙이 ROI 구간에 내포되거나
바운딩 박스의 중앙으로부터 아래로 ¼ 지점이 ROI 구간에 내포되었을 때
객체가 인식되어 카운팅이 되게 끔 조건을 부여함.

AI 모델 학습 구조

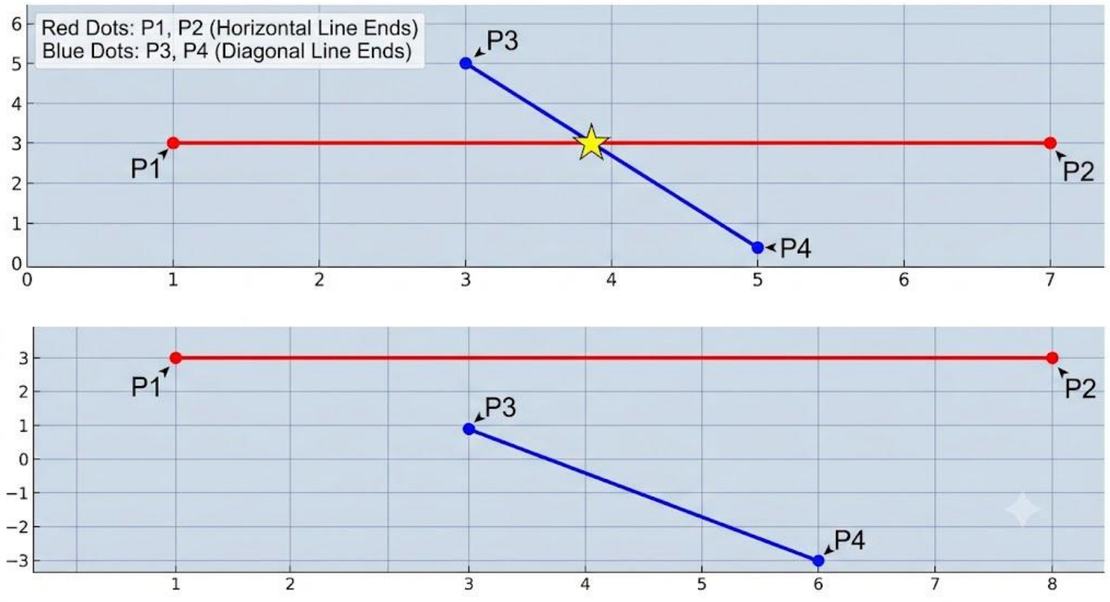
- 양옆 ROI(탐지 영역) 다각형 교차/겹침 검사 함수

```
def is_line_intersect(p1, p2, p3, p4):
    def ccw(A, B, C):
        return (C[1] - A[1]) * (B[0] - A[0]) > (B[1] - A[1]) * (C[0] - A[0])

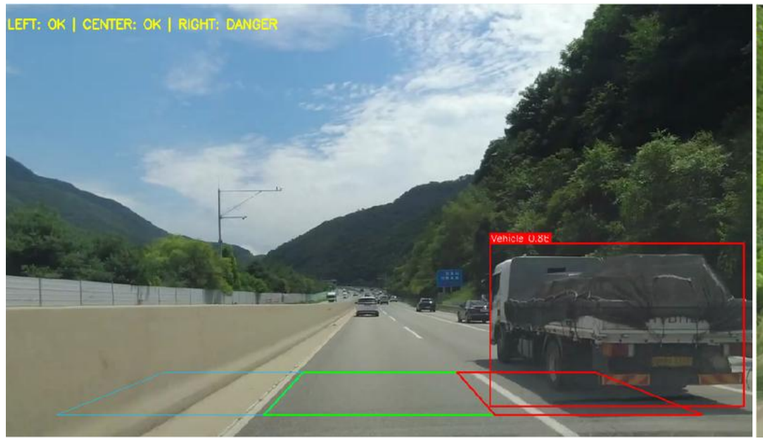
    return (ccw(p1, p3, p4) != ccw(p2, p3, p4)) and (
        ccw(p1, p2, p3) != ccw(p1, p2, p4)
    )
```

좌우의 탐지 영역의 경우에는
벡터의 외적(Cross Product)을 이용해 **A → B 방향 벡터** 기준으로 **점 C**가 어느 영역에 있는지 확인하여 True / False를 반환

선분 P1-P2(바운딩 박스 변)를 연장한 직선을 기준으로, 점 P3과 점 P4 (ROI 변의 양 끝점)가 서로 반대 방향에 위치하는지 확인



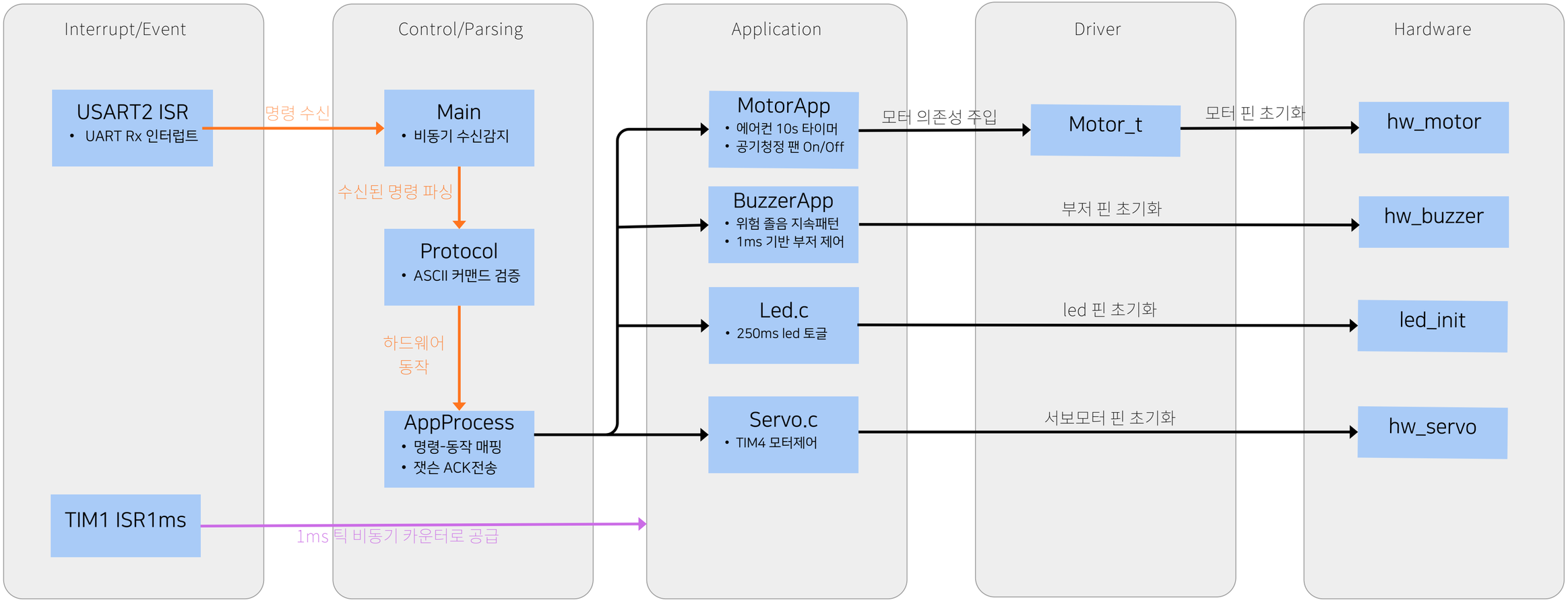
AI 모델 학습 구조


LEFT: OK | CENTER: OK | RIGHT: DANGER
Vehicle 0.8


LEFT: OK | CENTER: OK | RIGHT: OK
[EVENT] Tunnel Entrance Detected!
Tunnel Entrance 0.8


LEFT: OK | CENTER: OK | RIGHT: OK
[EVENT] Tunnel Exit Detected!
Tunnel Exit 0.75

STM32 소프트웨어 아키텍처

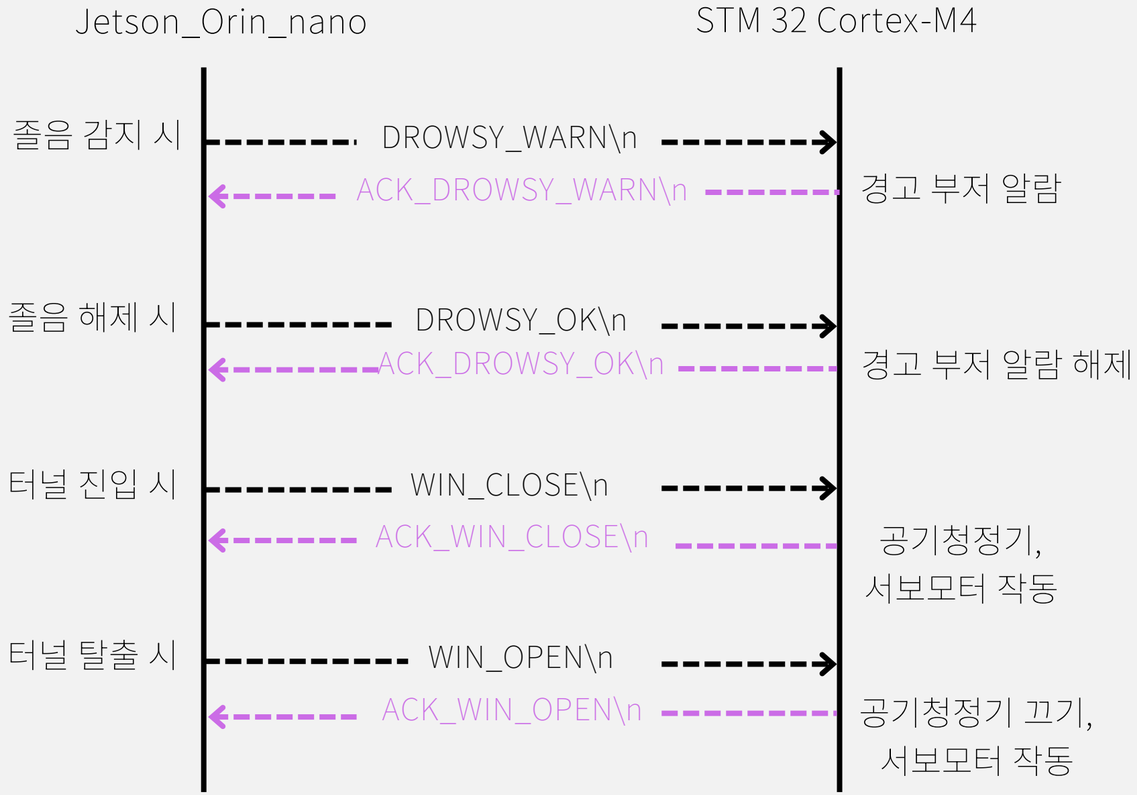


UART 통신 인터페이스 & 프로토콜 규격

Baud Rate: 115200 bps
Data Frame: Start 1-bit + Data 8-bit + Stop 1-bit
End-of-Line: ₩n

양방향 Handshake (ACK 응답 메커니즘)
- Jetson 명령 수신 및 하드웨어 구동 직후 즉각 ACK_<CMD> 회신
- 패킷 유실 방지 및 Jetson 단에서의 명령 수신 여부 화면 로그 검증

스트링 기반 패킷 구조
- 디버깅 직관성 및 가독성 확보를 위해 텍스트 기반 커맨드 채택
- 종단 문자(₩n) 기반 수신 버퍼 처리로 패킷 뭉침 현상 방지



프로토콜 송수신 및 데이터 무결성 검증

- 전 기능 시나리오 통신 검증: 부저, 에어컨, LED, 창문 제어커맨드의 정확한 송수신
- 데이터 무결성: 전송 문자열 및 16진수의 패킷 일치 확인
- 예외 처리: 오타 및 미정의 명령에 대한 즉각적인 NACK 피드백 구현

```
(venv) aidl@ubuntu:~/work$ python test.py
[SYSTEM] 포트 연결 완료: /dev/ttyACM0 (115200bps)

--- [1] 졸음 부저 테스트 ---
[TX] DROWSY_WARN
[RX] [13B] HEX: 44 52 4F 57 53 59 5F 57 41 52 4E 0D 0A | TXT: 'DROWSY_WARN\r\n'
[TX] DROWSY_OK
[RX] [10B] HEX: 44 52 4F 57 53 59 5F 4F 4B 0D | TXT: 'DROWSY_OK\r'
[RX] [1B] HEX: 0A | TXT: '\n'

--- [2] 졸음 에어컨 제어 테스트 ---
[TX] VENT_ON
[RX] [9B] HEX: 56 45 4E 54 5F 4F 4E 0D 0A | TXT: 'VENT_ON\r\n'

--- [3] LED 경고 켜기 테스트 ---
[TX] WARN_CENTER
[RX] [13B] HEX: 57 41 52 4E 5F 43 45 4E 54 45 52 0D 0A | TXT: 'WARN_CENTER\r\n'
[TX] WARN_RIGHT
[RX] [12B] HEX: 57 41 52 4E 5F 52 49 47 48 54 0D 0A | TXT: 'WARN_RIGHT\r\n'
[TX] WARN_LEFT
[RX] [11B] HEX: 57 41 52 4E 5F 4C 45 46 54 0D 0A | TXT: 'WARN_LEFT\r\n'

--- [4] LED 끄기 테스트 ---
[TX] CENTOR_OK
[RX] [18B] HEX: 4E 41 43 4B 5F 49 4E 56 41 4C 49 44 5F 43 4D 44 0D 0A | TXT: 'NACK_INVALID_CMD\r\n'
[TX] RIGHT_OK
[RX] [10B] HEX: 52 49 47 48 54 5F 4F 4B 0D 0A | TXT: 'RIGHT_OK\r\n'
[TX] LEFT_OK
[RX] [9B] HEX: 4C 45 46 54 5F 4F 4B 0D 0A | TXT: 'LEFT_OK\r\n'

--- [5] 창문 및 공기청정기 제어 테스트 ---
[TX] WIN_CLOSE
[RX] [11B] HEX: 57 49 4E 5F 43 4C 4F 53 45 0D 0A | TXT: 'WIN_CLOSE\r\n'
[TX] WIN_OPEN
[RX] [10B] HEX: 57 49 4E 5F 4F 50 45 4E 0D 0A | TXT: 'WIN_OPEN\r\n'

--- [6] 예외 커맨드 테스트 ---
[TX] INVALID_CMD
[RX] [18B] HEX: 4E 41 43 4B 5F 49 4E 56 41 4C 49 44 5F 43 4D 44 0D 0A | TXT: 'NACK_INVALID_CMD\r\n'
[SYSTEM] 통신 종료
```

기술적 문제 및 트러블 슈팅

문제 발생

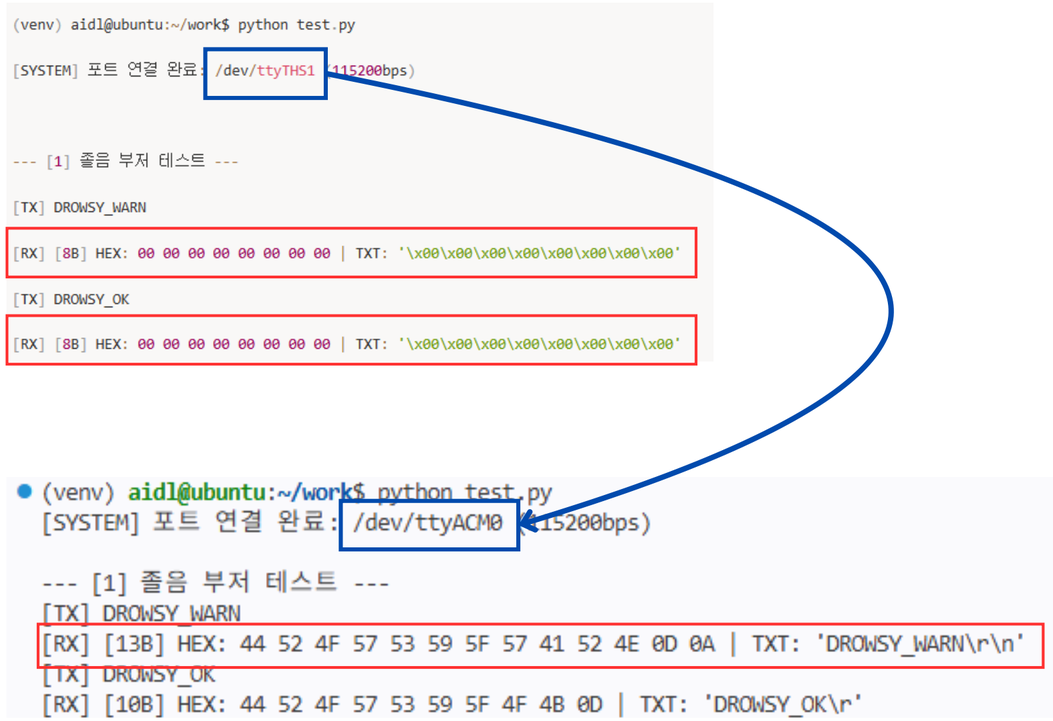
Jetson 40-pin GPIO (/dev/ttyTHS1)로 8바이트 이상 수신 시
데이터가 0x00으로 깨지거나 유실됨

원인 분석

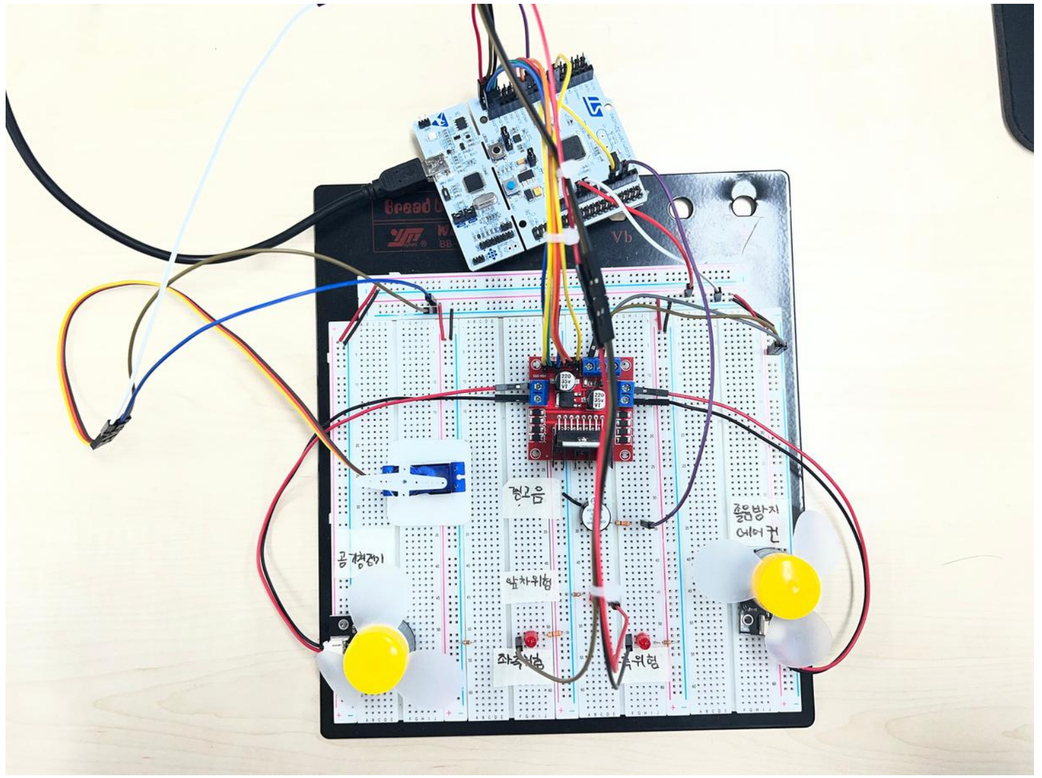
Jetson Orin Nan(JetPack 6.x / 현재 6.23 버전 기준)
UART 드라이버의 DMA 버그로
7바이트 이하는 정상작동하나 8바이트 부터 활성화되는
DMA 컨트롤러가 JetPack 6.x 디바이스 트리 매핑 결함으로 인해
SMMU가 메모리 쓰기 차단됨에 따라 수신받지 못함.
NVIDIA 공식 질의 응답 사이트에 보고되어있음

해결 방안

통신 경로를 USB-to-UART인터페이스(STM32 USART2)로 우회 변경하여 8-bit 데이터 무결성 및 장문 패킷 통신 신뢰성 확보



사진



사용 부품



Jetson Orin Nano
카메라 영상 분석 및 명령 전송



STM32
Jetson 명령 수신 및 하드웨어 제어



USB 카메라
운전자 촬영



서보모터
창문 구동



부저
졸음 및 차량 근접 경고



모터 드라이버
DC 모터의 방향과 동작 제어

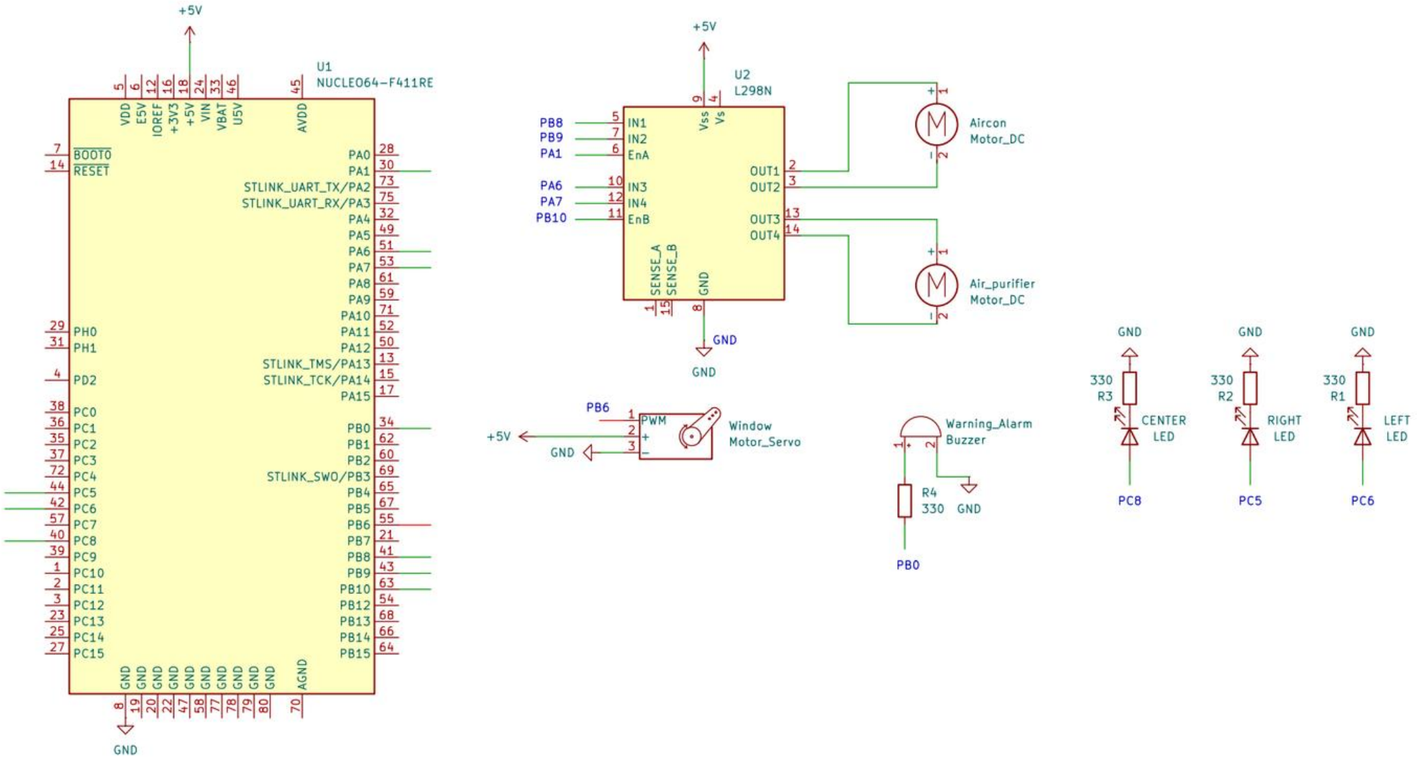


DC 모터
공기 청정기 구동

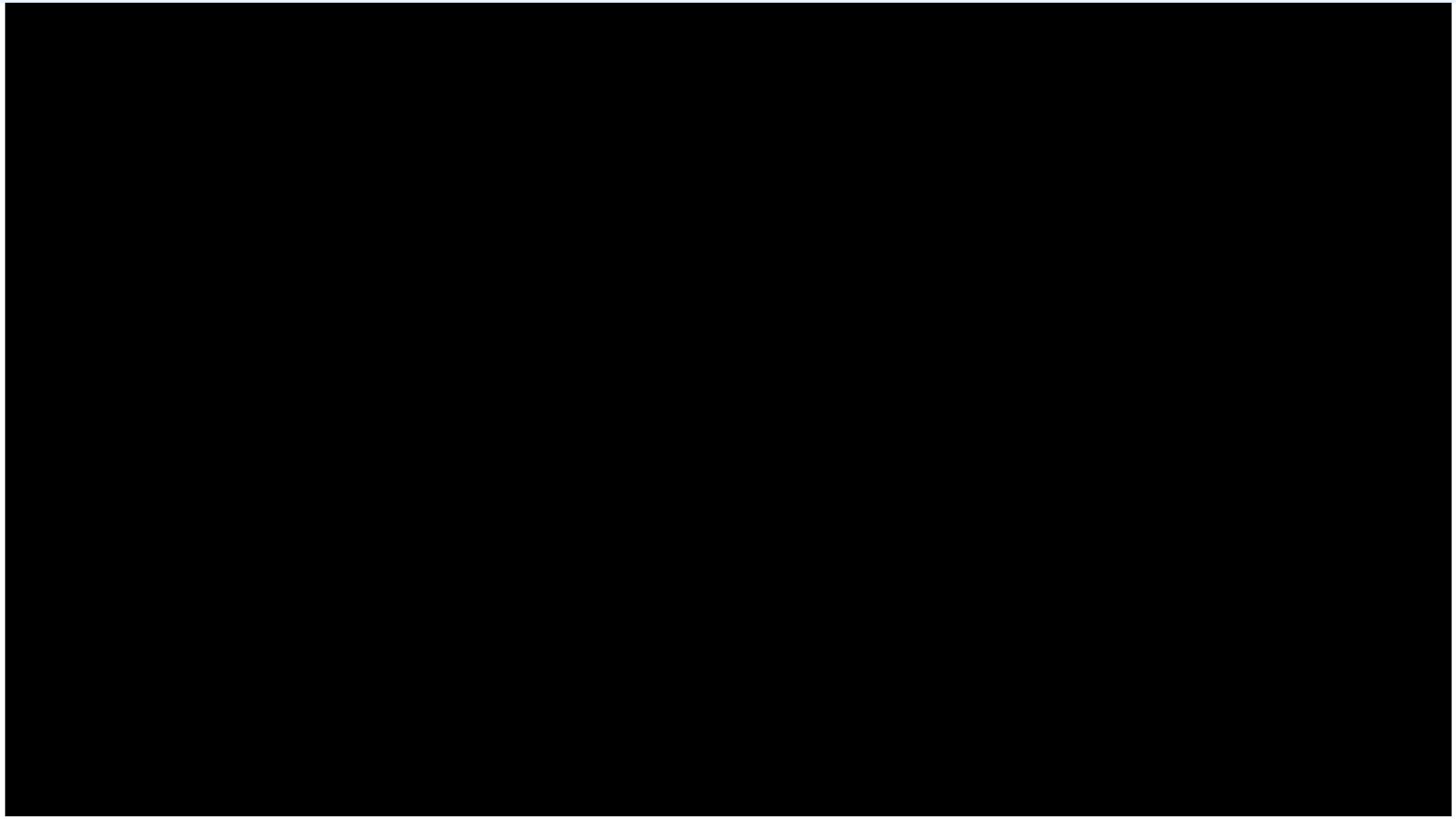


LED
접근 차량의 방향 표시

회로도


+5V
U1
NUCLEO64-F411RE
VDD
E5V
IOREF
+3V3
+5V
VIN
VBAT
U5V
AVDD
BOOT0
RESET
PA0
PA1
STLINK_UART_TX/PA2
STLINK_UART_RX/PA3
PA4
PA5
PA6
PA7
PA8
PA9
PA10
PA11
PA12
STLINK_TMS/PA13
STLINK_TCK/PA14
PA15
PH0
PH1
PD2
PC0
PC1
PC2
PC3
PC4
PC5
PC6
PC7
PC8
PC9
PC10
PC11
PC12
PC13
PC14
PC15
PB0
PB1
PB2
STLINK_SWO/PB3
PB4
PB5
PB6
PB7
PB8
PB9
PB10
PB12
PB13
PB14
PB15
GND
AGND
U2
L298N
PB8
PB9
PA1
PA6
PA7
PB10
IN1
IN2
EnA
IN3
IN4
EnB
Vss
Vs
OUT1
OUT2
OUT3
OUT4
SENSE_A
SENSE_B
Aircon
Motor_DC
Air_purifier
Motor_DC
PB6
PWM
Window
Motor_Servo
Warning_Alarm
Buzzer
R4
330
330
R3
CENTER
LED
PC8
330
R2
RIGHT
LED
PC5
330
R1
LEFT
LED
PC6

시연영상



프로젝트 후기

- EAR 임계값을 설정하는 과정에서, 실제로 눈을 감지 않았는데도 감은 것으로 오인하는 경우가 발생하여 적절한 임계값을 찾는데 어려움을 겪었다. 하지만 여러 번의 실험으로 적절한 값을 찾아내었다.

- ROI 탐지 방식을 채택한 초반에 후방에서 전방으로 나오는 차량에 대한 인식이 느린 문제가있었다. 그래서 검출방식을 기존 검출박스에 하단부 중앙이 들어오는 조건에서 적절한 조건으로 조치하여 해당 문제를 해결하였다.

감사합니다



https://github.com/takecareYata/jojimara.git